Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

SIAM J. SCI. COMPUT.
Vol. 48, No. 4, pp. C1011–C1036
© 2026 Society for Industrial and Applied Mathematics

ACCELERATING NEURAL NETWORK-BASED REGRESSION AND CLASSIFICATION TASKS THROUGH EMPIRICAL KERNELS*

SAAD QADEER†, ANDREW ENGEL‡, AMANDA HOWARD†, ADAM TSOU§, MAX VARGAS†, PANOS STINIS¶, AND TONY CHIANG‖

Abstract. The impressive performance of deep neural networks (DNNs) on a variety of learning tasks has spurred much investigation into improving their training and characterizing the trained functions. Recent work has shown the equivalence of DNNs in the infinite width limit and kernel machines relying on the neural tangent kernel (NTK) at initialization. These results suggest, and experimental evidence corroborates, that kernel machines relying on empirical kernels extracted from trained DNNs can act as surrogates for trained finite-width DNNs. The high computational cost of assembling the NTK, however, makes this approach infeasible in practice. In the current work, we study the performance of the conjugate kernel (CK), an efficient approximation to the NTK. For smooth function and logistic regression, we show that the CK performance is only marginally worse than that of the NTK and, in certain cases, much more superior. In particular, we establish bounds for the test losses, verify them with numerical tests, and identify the regularity of the kernel as the key determinant of performance. We also determine regimes where both kernel machines trained on features extracted from an underlying DNN are demonstrably superior to the latter and use this to suggest a recipe for accelerating DNN performance inexpensively. We present a demonstration of this on foundation models by comparing their performance on a classification task using a conventional technique and our prescription. We also show how our approach can be used to improve physics-informed operator network training as well as convolutional neural network training for vision classification tasks.

Key words. empirical kernel machines, generalization errors of finite-width neural networks, foundation models

MSC codes. 65D15, 68T07, 62J12

DOI. 10.1137/25M1731794

*Submitted to the journal's Machine Learning Methods for Scientific Computing section February 21, 2025; accepted for publication (in revised form) February 18, 2026; published electronically August 6, 2026.

https://doi.org/10.1137/25M1731794

Funding: The work of the first author was supported by the Department of Energy (DOE) Office of Advanced Scientific Computing Research (ASCR) through the Pacific Northwest National Laboratory Distinguished Computational Mathematics Fellowship (project 71268). The work of the second, fourth, fifth, and seventh authors were partially supported by the Mathematics for Artificial Reasoning in Science (MARS) initiative via the Laboratory Directed Research and Development (LDRD) Program at PNNL. The work of the third and sixth authors were partially supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research Program under the Scalable, Efficient and Accelerated Causal Reasoning Operators, Graphs and Spikes for Earth and Embedded Systems (SEA-CROGS) project (80278). The computational work was partially performed using PNNL Institutional Computing. The Pacific Northwest National Laboratory is a multi-program national laboratory operated for the U.S. Department of Energy by Battelle Memorial Institute under contract DE-AC05-76RL01830.

†Pacific Northwest National Laboratory, Richland, WA 99354 USA (saad.qadeer@pnnl.gov, amanda.howard@pnnl.gov, max.vargas@pnnl.gov).

‡Pacific Northwest National Laboratory, Richland, WA 99354 USA, and Department of Physics, The Ohio State University, Columbus, OH 43210 USA (andrew.engel@pnnl.gov).

§Pacific Northwest National Laboratory, Richland, WA 99354 USA, and Johns Hopkins University, Baltimore, MD 21218 USA (adamyuantsou@gmail.com).

¶Pacific Northwest National Laboratory, Richland, WA 99354 USA, Department of Applied Mathmatics, University of Washington, Seattle, WA 98195 USA, and Division of Applied Mathematics, Brown University, Providence, RI 02912 USA (panagiotis.stinis@pnnl.gov).

‖Pacific Northwest National Laboratory, Richland, WA 99354 USA, and Department of Mathematics, University of Washington, Seattle, WA 98195 USA (tonychiang.phd@gmail.com).

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

1. Introduction. Deep neural networks (DNNs) have shown immense promise in performing a variety of learning tasks, including, among others, image classification [11, 19], natural language processing [16, 17, 26, 37], function approximation [5, 41], solving differential equations [15, 18, 24], etc. Even though a theoretical understanding of their effectiveness and limitations remains elusive, partly down to their somewhat unwieldy architectures and complex loss landscapes, they continue to find wide usage and spur further investigation into improving their training and characterizing the optimally learned functions.

Kernel machines using the neural tangent kernel (NTK) for a DNN are known to furnish such a characterization in the infinite width limit [14]. Generally, kernel machines implicitly couple a linear method with a high-dimensional feature space embedding, classically with fixed kernel functions [4, 25, 34]. They have been well studied historically and widely deployed due to their efficiency, interpretable dependence on training data, and amenability to analysis. However, their reliance on a fixed kernel function somewhat limits their ability to learn features particular to the dataset, although some progress on developing learnable kernels has been made recently [32].

The NTK for a DNN is defined as the kernel function corresponding to the feature map given by the Jacobian of the DNN with respect to all the trainable parameters [14]. Recent work has established that randomly initialized DNNs in the infinite width limit converge to kernel machines relying on a deterministic NTK that does not deviate from its initial form [1, 20]. As a consequence, the result of fully training an infinite width DNN can be a priori identified as a kernel machine employing the known NTK at initialization. These insights also allow one to study the performance of the DNN architecture in the extremal overparameterized limit and benefit from the double-descent phenomenon [2, 3, 22].

It should be stressed that this limiting regime does not account for the changes in the DNN during training and hence fails to make use of features learned by the architecture. However, these results suggest that a DNN in the finite width regime could be well approximated by a kernel machine relying on the empirical NTK, i.e., the kernel corresponding to the Jacobian extracted from a trained DNN. As shown elsewhere [7, 23] as well as later in this article, under certain assumptions this claim is not only corroborated by strong evidence, both mathematical ((3.3) and sections 4 and 5) and experimental (section 6) but also considerably improved upon. This enables a precise determination of the better trained functions and in principle allows a more thorough study of their performance on unseen data.

The high computational cost of assembling the complete NTK for DNNs typically employed in practical applications [27, 28], however, makes this approach infeasible, motivating the need for low cost approximations. In this article, we study the performance of the conjugate kernel (CK) [8, 23], a "zeroth-order" approximation to the NTK, that has been observed to yield fairly similar results [7, 23] (see also section 6). For the problems of regression of smooth functions and logistic regression, we prove that the CK approximations are only marginally worse than the NTK approximations and, in some cases, are much more superior. In particular, we establish bounds for the errors, verify them with numerical tests, and identify the regularity of the kernel as the key determinant of performance. We also determine regimes where both kernel machines trained on features extracted from an underlying DNN are demonstrably superior to the latter. In addition to providing a theoretical grounding for replacing NTKs with CKs, our framework provides insights into understanding the robustness of the various approximations and suggests a recipe for boosting a DNN and a

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

physics-informed Deep Operator Network (DeepONet) [24, 38] accuracy inexpensively. We provide a demonstration of this on GPT-2 by comparing its performance on a classification task using a conventional approach and our prescription (see section 7).

We emphasize again that the empirical kernels we study are calculated from trained DNNs and not the randomly initialized networks. In the literature, our objects of study have been referred to variously as the after kernel [23] and the data-dependent NTK [9] (for the NTK) and the conjugate projection (for the CK, which is not to be confused with a neural network Gaussian process, which refers to the CK at initialization) [23]. It follows that our results do not advocate for a wholesale replacement of DNNs by kernel machines, as that would fail to leverage the remarkable training properties of the DNNs but rather prescribe parallel or postprocessing operations that yield learned functions with improved accuracy and better interpretability.

2. Preliminaries. In this section, we introduce our notation and define the basic terms and tools. We denote by $f_{\mathrm{NN}} : \mathbb{R}^{d_0} \to \mathbb{R}^{d_{L+1}}$ a fully connected network (FCN) with L hidden layers of the form

$$\begin{aligned} \boldsymbol{z}^{(l)} &= \sigma\left(W^{(l)}\boldsymbol{z}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad 1 \le l \le L, \\ \boldsymbol{z}^{(L+1)} &= W^{(L+1)}\boldsymbol{z}^{(L)} + \mathbf{b}^{(L+1)}. \end{aligned} \tag{2.1}$$

Here, σ is the activation function, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$ are the trainable parameters, $\boldsymbol{z}^{(0)} \in Z$ is the input, and $\boldsymbol{z}^{(L+1)} \in \mathbb{R}^{d_{L+1}}$ is the output, i.e., $f_{\mathrm{NN}}(\boldsymbol{z}^{(0)}) = \boldsymbol{z}^{(L+1)}$. We denote all the trainable parameters $\{W^{(l)}, \mathbf{b}^{(l)}\}_{1 \le l \le L+1}$ by θ and their number by $|\theta|$ for brevity, and we indicate the dependence of the neural network on them by writing f^{θ}_{NN}. Since this is the only type of neural network (NN) we shall consider in this article (barring some later examples), henceforth we refer to FCNs of this type as NNs.

Architectures of this type can be deployed for various tasks. For instance, let $f : Z \to \mathbb{R}$ be a given function, where Z is a compact subset of $\mathbb{R}^{d_0}$, with training input-output pairs $\{(\boldsymbol{x}_i, y_i)\}_{0 \le i \le N} \subset Z \times \mathbb{R}$ (so that $y_i = f(\boldsymbol{x}_i)$). In order to solve the function regression problem, the parameters can be trained so as to minimize the weighted squared loss

$$L_{0,\mathrm{NN}}(\theta) = \sum_{i=0}^{N} w_i \left| y_i - f^{\theta}_{\mathrm{NN}}(\boldsymbol{x}_i) \right|^2, \tag{2.2}$$

for some nonnegative weights $\{w_i\}_{0 \le i \le N}$, and yield an approximation $f^{\theta^*}_{\mathrm{NN}}$ to the target function. We further define the seminorms

$$\|g\|_0 := \left(\sum_{i=0}^{N} w_i \left| g(\boldsymbol{x}_i) \right|^2 \right)^{1/2}, \qquad \|g\|_1 := \left(\sum_{i=0}^{M} v_i \left| g(\boldsymbol{y}_i) \right|^2 \right)^{1/2} \tag{2.3}$$

for any $g : Z \to \mathbb{R}^{d_{L+1}}$, where $\{(\boldsymbol{y}_i, v_i)\}_{0 \le i \le M}$ is a collection of test nodes and associated nonnegative weights. The training and test errors are then $\|f - f^{\theta^*}_{\mathrm{NN}}\|_0 = [L(\theta^*)]^{1/2}$ and $\|f - f^{\theta^*}_{\mathrm{NN}}\|_1$, respectively.

Similarly, we can employ NNs for classifications tasks. Let $\{(\boldsymbol{x}_i, \chi_i)\}_{0 \le i \le N} \subset Z \times \{0, 1\}$ be the training set, where χ_i is the class label assigned to $\boldsymbol{x}_i$. Set $d_{L+1} = 2$ (so that $f^{\theta}_{\mathrm{NN}} : \mathbb{R}^{d_0} \to \mathbb{R}^2$), and apply the softmax function to get

$$p^{\theta}_{\mathrm{NN}}(\boldsymbol{z}) = \frac{\exp\left(f^{\theta}_{\mathrm{NN},1}(\boldsymbol{z})\right)}{\sum_{i=1}^{2} \exp\left(f^{\theta}_{\mathrm{NN},i}(\boldsymbol{z})\right)} = \frac{1}{1 + \exp\left[-\left(f^{\theta}_{\mathrm{NN},1}(\boldsymbol{z}) - f^{\theta}_{\mathrm{NN},2}(\boldsymbol{z})\right)\right]}. \tag{2.4}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

This is the likelihood, according to this model, of $\boldsymbol{z} \in Z$ belonging to the class labeled 1. The training requires the minimization of the logistic loss (i.e., the cross-entropy)

$$\ell_{0,\mathrm{NN}}(\theta) = -\sum_{i=0}^{N} \chi_i \ln\left(p^{\theta}_{\mathrm{NN}}(\boldsymbol{x}_i)\right) + (1-\chi_i)\ln\left(1-p^{\theta}_{\mathrm{NN}}(\boldsymbol{x}_i)\right), \tag{2.5}$$

with the corresponding test loss

$$\ell_{1,\mathrm{NN}}(\theta) = -\sum_{i=0}^{M} \mu_i \ln\left(p^{\theta}_{\mathrm{NN}}(\boldsymbol{y}_i)\right) + (1-\mu_i)\ln\left(1-p^{\theta}_{\mathrm{NN}}(\boldsymbol{y}_i)\right), \tag{2.6}$$

where $\{\mu_i\}_{0\le i\le M} \subset \{0,1\}$ are the class labels of the test nodes.

These tasks can also be performed with the aid of kernel methods [4, 25, 34]. Given a valid symmetric and positive-definite kernel function $\ker : Z \times Z \to \mathbb{R}$, define the kernel matrix $H_{\ker} = \left(\ker(\boldsymbol{x}_i, \boldsymbol{x}_j)\right)_{0\le i,j\le N}$, and the weighted kernel matrix $\widehat{H}_{\ker} = W^{1/2} H_{\ker} W^{1/2}$, where $W^{1/2} = \mathrm{diag}(\sqrt{w_0}, \ldots, \sqrt{w_N})$. The corresponding kernel approximation for the function regression problem above seeks $\boldsymbol{\delta}^{*,\ker} \in \mathbb{R}^{N+1}$ to minimize

$$L_{0,\ker}(\boldsymbol{\delta}) = \sum_{i=0}^{N} w_i \left| y_i - \sum_{j=0}^{N} \boldsymbol{\delta}_j \ker(\boldsymbol{x}_j, \boldsymbol{x}_i) \right|^2. \tag{2.7}$$

It possesses the closed-form solution

$$f_{\ker}(\boldsymbol{z}) = \boldsymbol{y}^{\top} W^{1/2} \left(\widehat{H}_{\ker}\right)^{\dagger} W^{1/2} \left(\ker(\boldsymbol{x}_j, \boldsymbol{z})\right)_{0\le j\le N}, \tag{2.8}$$

where $\boldsymbol{y} = \left(y_i\right)_{0<i<N}$ contains the training target values, with training and test losses $\|f - f_{\ker}\|_0$ and $\|f - f_{\ker}\|_1$, respectively.

The logistic regression problem can likewise be solved by finding an $\boldsymbol{\alpha}^{*,\ker} \in \mathbb{R}^{N+1}$ that minimizes

$$\ell_{0,\ker}(\boldsymbol{\alpha}) = \sum_{i=0}^{N} \chi_i \ln\left[1+\exp(-H_{\ker,(i,:)}\boldsymbol{\alpha})\right] + (1-\chi_i)\ln\left[1+\exp(H_{\ker,(i,:)}\boldsymbol{\alpha})\right], \tag{2.9}$$

thus yielding the likelihood

$$p_{\ker}(\boldsymbol{z}) = \left[1+\exp\left(-\sum_{j=0}^{N} \ker(\boldsymbol{z}, \boldsymbol{x}_j)\boldsymbol{\alpha}_j^{*,\ker}\right)\right]^{-1}, \tag{2.10}$$

according to this classifier, of $\boldsymbol{z} \in Z$ lying in class 1. The test loss $\ell_{1,\ker}$ is defined similarly to (2.6) with (2.4) replaced by (2.10).

In case ker is induced by a feature map $\Phi_{\ker} : Z \to \mathbb{R}^K$, so that $\ker(\boldsymbol{x}, \boldsymbol{z}) = \Phi_{\ker}(\boldsymbol{x})^{\top}\Phi_{\ker}(\boldsymbol{z})$, (2.8) can be understood as an orthogonal projection on $\mathcal{S}_{\ker} = \mathrm{span}(\{\Phi_{\ker,k}\}_{k=1}^{K})$ with respect to $\langle\cdot,\cdot\rangle_0$, the inner product that induces $\|\cdot\|_0$; a technique for obtaining an optimal representation of this projection is presented in section SM4 of the supplementary materials (M173179_Supplementary_Materials.pdf

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[local/web 301KB]). Equivalently, we can interpret (2.8) as weighted linear regression on the data-points $\{(\Phi_{\ker}(\boldsymbol{x}_i), y_i)\}_{0\le i\le N}$; i.e., it implicitly finds $\boldsymbol{\gamma}^{*,\ker} \in \mathbb{R}^K$, which minimizes the training loss

$$\widetilde{L}_{0,\ker}(\boldsymbol{\gamma}) = \sum_{i=0}^{N} w_i \left| y_i - \boldsymbol{\gamma}^\top \Phi_{\ker}(\boldsymbol{x}_i) \right|^2, \tag{2.11}$$

so the approximation (2.8) can also be written as

$$f_{\ker}(\boldsymbol{z}) = \left(\boldsymbol{\gamma}^{*,\ker}\right)^\top \Phi_{\ker}(\boldsymbol{z}). \tag{2.12}$$

This equivalence between the apparently more general form $\boldsymbol{\gamma}^\top \Phi_{\ker}(\boldsymbol{z})$ in (2.12) and the kernel form $\boldsymbol{\delta}^\top \left(\ker(\boldsymbol{x}_j, \boldsymbol{z})\right)_{0\le j\le N}$ in (2.8) can be viewed as a consequence of the representer theorem [33] or, equivalently, as a corollary of the (easily proven) fact that

$$\operatorname{range}(A^\top A) = \operatorname{range}(A^\top) \tag{2.13}$$

for any matrix A: applied to the weighted feature map matrix $\widehat{\Xi}_{\ker} = (\Phi_{\ker,m}(\boldsymbol{x}_i)\, w_i^{1/2})_{1\le m\le K, 0\le i\le N} \in \mathbb{R}^{K\times(N+1)}$, we deduce that for an optimal $\boldsymbol{\gamma}^{*,\ker}$, there exists some $\boldsymbol{\delta}^{*,\ker} \in \mathbb{R}^{N+1}$ such that

$$\widehat{\Xi}_{\ker}^\top \boldsymbol{\gamma}^{*,\ker} = \widehat{\Xi}_{\ker}^\top \widehat{\Xi}_{\ker} \boldsymbol{\delta}^{*,\ker} = \widehat{H}_{\ker} \boldsymbol{\delta}^{*,\ker}, \tag{2.14}$$

since $\widehat{H}_{\ker} = \widehat{\Xi}_{\ker}^\top \widehat{\Xi}_{\ker}$, and hence that we can restrict our search for $\boldsymbol{\gamma}^{*,\ker}$ in $\operatorname{range}(\widehat{\Xi}_{\ker})$. This implies that

$$\|f - f_{\ker}\|_\nu^2 = \widetilde{L}_{\nu,\ker}(\boldsymbol{\gamma}^{*,\ker}), \tag{2.15}$$

for $\nu = 0, 1$, where the test loss $\widetilde{L}_{1,\ker}$ is defined in the same manner as (2.11) with the test points and weights replacing the training ones.

Similarly, solving the kernel logistic problem can be interpreted as a linear classifier in the feature space, i.e., identifying the decision boundary

$$\left\{ \boldsymbol{x} \in \mathbb{R}^{d_0} : \left(\boldsymbol{\beta}^{*,\ker}\right)^\top \Phi_{\ker}(\boldsymbol{x}) = 0 \right\}, \tag{2.16}$$

which can be seen as a hyperplane in $\mathbb{R}^K$, by finding $\boldsymbol{\beta}^{*,\ker} \in \mathbb{R}^K$, which minimizes

$$\widetilde{\ell}_{0,\ker}(\boldsymbol{\beta}) = \sum_{i=0}^{N} \chi_i \ln\left[1 + \exp\left(-\boldsymbol{\beta}^\top \Phi_{\ker}(\boldsymbol{x}_i)\right)\right] + (1-\chi_i) \ln\left[1 + \exp\left(\boldsymbol{\beta}^\top \Phi_{\ker}(\boldsymbol{x}_i)\right)\right]. \tag{2.17}$$

Following an argument similar to that used above, we have

$$\ell_{\nu,\ker}(\boldsymbol{\alpha}^{*,\ker}) = \widetilde{\ell}_{\nu,\ker}(\boldsymbol{\beta}^{*,\ker}) \tag{2.18}$$

for $\nu = 0, 1$.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

3. Problem formulation. Given a (fully or partially trained or untrained) neural network $f_{\text{NN}} : \mathbb{R}^{d_0} \to \mathbb{R}$, the corresponding neural tangent kernel (NTK) is the kernel induced by the feature map

$$\Phi_{\text{NTK},k} := \frac{\partial f_{\text{NN}}^{\theta}}{\partial \theta_k}, \quad 1 \le k \le |\theta|, \tag{3.1}$$

i.e., the Jacobian of the NN with respect to the trainable parameters [14]. In the infinite width limit, it has been established that, over the course of training a randomly initialized NN, the NTK possesses an unchanging closed form and that the NN converges to the corresponding NTK machine [20]. This highlights the linear dependence of the NN on the underlying parameters, to wit,

$$f_{\text{NN}}^{\theta} = f_{\text{NN}}^{\theta_0} + (\theta - \theta_0)^{\top} \left(\frac{\partial f_{\text{NN}}^{\theta}}{\partial \theta_k} \right) \Big|_{\theta=\theta_0}. \tag{3.2}$$

In fact, since $f_{\text{NN}}^{\theta_0}$ is a linear combination of the neurons in the last hidden layer (from (2.1)) and which correspond to the Jacobian entries corresponding to the last layer of parameters, it can be subsumed in the second term in (3.2) to yield

$$f_{\text{NN}}^{\theta} = \boldsymbol{\gamma}^{\top} \Phi_{\text{NTK}} \tag{3.3}$$

for some $\boldsymbol{\gamma} \in \mathbb{R}^{|\theta|}$. Since this is precisely the form of an NTK approximation (from the equivalence of (2.8) and (2.12)), we deduce that, in the infinite width limit, a fully trained NN is identical to an optimal regressor employing the NTK.

However, using this insight to obtain performance guarantees and assess limitations of finite width networks runs into two problems. First, the assumption of linear dependence on parameters in (3.2) does not hold [21], so the optimal NTK machine can only be viewed as the best approximator in the tangent space of the NN at θ_0 and not on the manifold of all NNs. Second, computing the NTK in practice is a computationally daunting task [27, 28]. In the infinite width regime and under random initialization, a closed form for the NTK can be calculated. However, empirical kernels do not possess such characterizations in general, necessitating the use of low-cost approximations.

The conjugate kernel (CK) is defined as the kernel induced by the Jacobian only with respect to the parameters $\{W^{(L+1)}, \mathbf{b}^{(L+1)}\}$ in the last layer [8, 23]. Consequently, its feature map can be seen as a truncation of the full Jacobian employed by the NTK. Observe that we have set $d_{L+1} = 1$ so $W^{(L+1)} \in \mathbb{R}^{d_L}$ and $\mathbf{b}^{(L+1)} \in \mathbb{R}$, and hence

$$\Phi_{\text{CK},k} := \begin{cases} \dfrac{\partial f_{\text{NN}}^{\theta}}{\partial W_k^{(L+1)}} & \text{if } 1 \le k \le d_L, \\ \dfrac{\partial f_{\text{NN}}^{\theta}}{\partial \mathbf{b}^{(L+1)}} & \text{if } k = d_L + 1. \end{cases} \tag{3.4}$$

A moment's thought then reveals that

$$\text{CK}(\boldsymbol{z}, \widetilde{\boldsymbol{z}}) := \Phi_{\text{CK}}(\boldsymbol{z})^{\top} \Phi_{\text{CK}}(\widetilde{\boldsymbol{z}}) = 1 + \boldsymbol{z}^{(L)} \cdot \widetilde{\boldsymbol{z}}^{(L)}. \tag{3.5}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Remark 3.1. For an NN trained for the binary classification problem described in the previous section, we have $d_{L+1} = 2$. In this setting, we define the NTK and CK for $f_{\text{diff}} := f^{\theta}_{\text{NN},1} - f^{\theta}_{\text{NN},2}$ since, as shown in (2.4), this difference dictates the output of the logistic predictor.

Remark 3.2. Since the trained NN function can be obtained by attaching the learned weights in the last hidden layer to the CK feature map, i.e.,

$$f^{\theta}_{\text{NN}}(\boldsymbol{z}) = \begin{pmatrix} W^{(L+1)} & \mathbf{b}^{(L+1)} \end{pmatrix} \Phi_{\text{CK}}(\boldsymbol{z}), \tag{3.6}$$

the optimal CK approximator (and hence the NTK approximator) must necessarily outperform the NN on the training set.

Denoting by E the contribution towards the NTK from every layer but the last allows us to write

$$\text{NTK}(\boldsymbol{z}, \widetilde{\boldsymbol{z}}) = \text{CK}(\boldsymbol{z}, \widetilde{\boldsymbol{z}}) + \text{E}(\boldsymbol{z}, \widetilde{\boldsymbol{z}}). \tag{3.7}$$

The CK can therefore be viewed as a "zeroth-order" approximation to the NTK. All these components are valid kernels that can be calculated from a given NN. However, the cost of computing the CK at a pair of points is the same as that of evaluating the NN at those points since, according to (3.5), it makes use of the neuron values at the last hidden layer and hence is significantly lower than computing the NTK. Moreover, numerical evidence suggests that, while the NTK outperforms the CK on test sets, the improvement is only marginal and is dwarfed by the superiority both kernels exhibit over the NN architecture from which they are derived [7, 23] (also see section 6). In other words, the CK appears to yield substantive gains in accuracy over a trained finite width NN while being easier to calculate than the NTK and much cheaper to train further than the NN. This raises the intriguing possibility that the CK can serve as a low-cost proxy for the NTK, i.e., lead to a comparable boost in performance over the NN while being significantly less expensive to assemble than the NTK. Since training the CK is the same as further training the last parameter layer of the NN, our work provides a recipe for boosting NN accuracy fairly inexpensively and sheds light on the robustness (or lack thereof) conferred on the approximations by the choice of the activation function.

In the following sections, we provide a mathematical treatment of this phenomenon for two types of problems: regression of smooth functions and classification using logistic regression. For ease of exposition and analysis, we concentrate on these problems in low dimensions with structured training and test nodes. The analyses of these problems are presented in sections 4 and 5, and are complemented by numerical experiments presented in sections 6 and 7, ranging from toy models to physics-informed operator learning, computer vision classification, and large language models.

4. Function regression.

4.1. Preliminaries. We consider the approximation problem for a smooth function $f : [a, b] \to \mathbb{R}$ whose values are known at the equispaced training nodes $x_j = a + j\Delta x$ for $0 \le j \le N$, where $\Delta x = (b - a)/N$. The performance is assessed on the

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

equispaced test nodes $y_j = a + j\Delta y$ for $0 \le j \le M$, where $\Delta y = (b-a)/M$; we take $M = \tau N$, for some integer $\tau > 1$, and set

$$w_i = \begin{cases} \Delta x, & 1 \le i \le N-1, \\ \Delta x/2, & i = 0, N, \end{cases} \qquad v_i = \begin{cases} \Delta y, & 1 \le i \le M-1, \\ \Delta y/2, & i = 0, M, \end{cases} \tag{4.1}$$

so for any $g : [a,b] \to \mathbb{R}$, we have

$$\|g\|_0 = \left(\frac{\Delta x}{2} \sum_{j=0}^{N-1} \left(g(x_j)^2 + g(x_{j+1})^2 \right) \right)^{1/2}, \quad \|g\|_1 = \left(\frac{\Delta y}{2} \sum_{j=0}^{M-1} \left(g(y_j)^2 + g(y_{j+1})^2 \right) \right)^{1/2}. \tag{4.2}$$

These can be recognized as the trapezoidal rule approximations to the L^2 norm on $[a,b]$; recall that this approximation is second-order accurate with respect to the step-size in general and spectrally accurate for periodic functions. Moreover, these choices serve to considerably simplify the analysis: for instance, since $y_{\tau j} = x_j$ for $0 \le j \le N$, we have, for any function g,

$$\|g\|_0^2 = \frac{\Delta x}{2} \sum_{j=0}^{N-1} \left(g(x_j)^2 + g(x_{j+1})^2 \right) \le \frac{\Delta x}{2} \sum_{j=0}^{N-1} \sum_{i=0}^{\tau} g(y_{\tau j+i})^2 = \left(\frac{M}{N} \right) \|g\|_1^2$$

so that

$$\|g\|_0 \le \tau^{1/2} \|g\|_1 . \tag{4.3}$$

This straightforward result allows us to describe the size of the function on the lower-resolution training grid in terms of the higher-resolution test grid. The next two results enable us to move in the opposite direction, i.e., bounding the seminorm on the finer grid by that on the coarser grid. This naturally requires some assumptions about the behavior of the function away from the training nodes: the following lemmas consider two different settings; the proofs are provided in section SM1 of the supplementary materials.

LEMMA 4.1. *Suppose that g is monotonic on every subinterval $[x_i, x_{i+1}]$. Then*

$$\|g\|_1 \le \sqrt{2} \|g\|_0 . \tag{4.4}$$

Combining this with (4.3), we have the equivalence

$$\tau^{-1/2} \|g\|_0 \le \|g\|_1 \le \sqrt{2} \|g\|_0 . \tag{4.5}$$

LEMMA 4.2. *Suppose that g is Lipschitz on $[a,b]$ with Lipschitz constant L_g. Then*

$$\|g\|_1^2 \le 4 \left(\|g\|_0^2 + \frac{2(b-a)^2 L_g^2}{N^2} \right). \tag{4.6}$$

4.2. Approximation properties. For a smooth target function $f : [a,b] \to \mathbb{R}$, let f_{CK} and f_{NTK} be the CK and NTK approximations to f with respect to the training nodes. Recall that (2.8) can be viewed as an orthogonal projection $\mathcal{P}_{\mathrm{ker}}$ on

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

$\mathcal{S}_{\text{ker}}$, the span of the feature map components corresponding to the kernel. Using $\mathcal{S}_{\text{CK}} \subset \mathcal{S}_{\text{NTK}}$ and Remark 3.2, we have

$$\|f - f_{\text{NTK}}\|_0 \leq \|f - f_{\text{CK}}\|_0 \leq \|f - f_{\text{NN}}\|_0 . \tag{4.7}$$

Next, we establish that the error of the CK approximation may also be controlled by that of the NTK approximation for most target functions. More precisely, we define, for any $\beta \leq 1$,

$$\mathcal{F}_\beta = \{ f : [a,b] \to \mathbb{R} \text{ is smooth with } \|\mathcal{P}_{\text{NTK}} f\|_0 \leq \beta \|f\|_0 \} . \tag{4.8}$$

We note that since $\mathcal{P}_{\text{NTK}}$ is an orthogonal projection, its norm is one and so $\mathcal{F}_1$ contains all smooth real-valued functions. However, most functions of interest would not lie in $\mathcal{S}_{\text{NTK}}$ and hence would belong to some $\mathcal{F}_\beta$ with $\beta < 1$. For this class of functions, we have the following result.

LEMMA 4.3. *For any function $f \in \mathcal{F}_\beta$ with $\beta < 1$,*

$$\|f - f_{CK}\|_0 \leq \frac{1}{1-\beta} \|f - f_{NTK}\|_0 .$$

We then have the following approximation results.

THEOREM 4.1. *Suppose that $(f - f_{ker})$, where ker refers to the NTK or the CK, is monotonic on every subinterval $[x_i, x_{i+1}]$ for $0 \leq i \leq N-1$. Then*

$$\|f - f_{ker}\|_1 \leq \sqrt{2\tau} \|f - f_{NN}\|_1 . \tag{4.9}$$

Moreover, if $f \in \mathcal{F}_\beta$ for some $\beta < 1$ and both $(f - f_{CK})$ and $(f - f_{NTK})$ are monotonic on every subinterval $[x_i, x_{i+1}]$ for $0 \leq i \leq N-1$, then there exist constants $C_1, C_2 \geq 1$ such that

$$C_1^{-1} \|f - f_{NTK}\|_1 \leq \|f - f_{CK}\|_1 \leq C_2 \|f - f_{NTK}\|_1 . \tag{4.10}$$

Proof. We have

$$\begin{aligned} \|f - f_{\text{ker}}\|_1 &\leq \sqrt{2} \|f - f_{\text{ker}}\|_0 \qquad (\because \text{Lemma 4.1}) \\ &\leq \sqrt{2} \|f - f_{\text{NN}}\|_0 \qquad (\because (4.7)) \\ &\leq \sqrt{2\tau} \|f - f_{\text{NN}}\|_1 \qquad (\because (4.3)). \end{aligned} \tag{4.11}$$

A similar argument with $C_1 = \sqrt{2\tau}$ yields the first inequality in (4.10). Observe also that

$$\begin{aligned} \|f - f_{\text{CK}}\|_1 &\leq \sqrt{2} \|f - f_{\text{CK}}\|_0 \qquad (\because \text{Lemma 4.1}) \\ &\leq \frac{\sqrt{2}}{1-\beta} \|f - f_{\text{NTK}}\|_0 \qquad (\because \text{Lemma 4.3}) \\ &\leq \frac{\sqrt{2\tau}}{1-\beta} \|f - f_{\text{NTK}}\|_1 \qquad (\because (4.3)), \end{aligned} \tag{4.12}$$

so setting $C_2 = \frac{\sqrt{2\tau}}{1-\beta}$ yields the desired second inequality in (4.10). ∎

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Note that Theorem 4.1 requires the residuals to be monotonic, which may be too stringent. The following result rests on a more relaxed assumption.

THEOREM 4.2. *Suppose that $(f - f_{ker})$, where ker refers to the NTK or the CK, is Lipschitz with Lipschitz constant L_{ker}. Then*

$$\|f - f_{ker}\|_1^2 \leq 4\tau \|f - f_{NN}\|_1^2 + \frac{8(b-a)^2 L_{ker}^2}{N^2}. \tag{4.13}$$

Furthermore, if $f \in \mathcal{F}_\beta$ for some $\beta < 1$ and both $(f - f_{CK})$ and $(f - f_{NTK})$ are Lipschitz, with Lipschitz constants L_{CK} and L_{NTK}, then there exist constants $D_1, D_2 \geq 1$ such that

$$\|f - f_{NTK}\|_1^2 \leq D_1 \|f - f_{CK}\|_1^2 + \frac{8(b-a)^2 L_{NTK}^2}{N^2} \tag{4.14}$$

and

$$\|f - f_{CK}\|_1^2 \leq D_2 \|f - f_{NTK}\|_1^2 + \frac{8(b-a)^2 L_{CK}^2}{N^2}. \tag{4.15}$$

Proof. We have

$$\begin{aligned}
\|f - f_{\mathrm{ker}}\|_1^2 &\leq 4\left(\|f - f_{\mathrm{ker}}\|_0^2 + \frac{2(b-a)^2 L_{\mathrm{ker}}^2}{N^2}\right) && (\because \text{Lemma 4.2}) \\
&\leq 4\left(\|f - f_{\mathrm{NN}}\|_0^2 + \frac{2(b-a)^2 L_{\mathrm{ker}}^2}{N^2}\right) && (\because (4.7)) \\
&\leq 4\left(\tau \|f - f_{\mathrm{NN}}\|_1^2 + \frac{2(b-a)^2 L_{\mathrm{ker}}^2}{N^2}\right) && (\because (4.3)). \qquad (4.16)
\end{aligned}$$

Following an identical argument and setting $D_1 = 4\tau$ yields (4.14). In addition,

$$\begin{aligned}
\|f - f_{\mathrm{CK}}\|_1^2 &\leq 4\left(\|f - f_{\mathrm{CK}}\|_0^2 + \frac{2(b-a)^2 L_{\mathrm{CK}}^2}{N^2}\right) && (\because \text{Lemma 4.2}) \\
&\leq 4\left(\frac{1}{(1-\beta)^2}\|f - f_{\mathrm{NTK}}\|_0^2 + \frac{2(b-a)^2 L_{\mathrm{CK}}^2}{N^2}\right) && (\because \text{Lemma 4.3}) \\
&\leq 4\left(\frac{\tau}{(1-\beta)^2}\|f - f_{\mathrm{NTK}}\|_1^2 + \frac{2(b-a)^2 L_{\mathrm{CK}}^2}{N^2}\right) && (\because (4.3)), \qquad (4.17)
\end{aligned}$$

so $D_2 = \frac{4\tau}{(1-\beta)^2}$ yields (4.15). ∎

5. Logistic regression.

5.1. Preliminaries. In this section, we compare the performances of the NN and the two kernels under consideration for logistic regression. For ease of exposition, we consider the problem in two dimensions. Let $\{\boldsymbol{x}_{i,j}\}_{0\leq i\leq N_1, 0\leq j\leq N_2} \subset Z = [a_1, b_1] \times [a_2, b_2]$ be the training points given by

$$\boldsymbol{x}_{i,j} = (a_1 + i\Delta x_1, a_2 + j\Delta x_2),$$

where $\Delta x_1 = (b_1 - a_1)/N_1$ and $\Delta x_2 = (b_2 - a_2)/N_2$. We further set $h = (\Delta x_1^2 + \Delta x_2^2)^{1/2}$. The test nodes $\{\boldsymbol{y}_{i,j}\}_{0\leq i\leq M_1, 0\leq j\leq M_2}$ are similarly chosen as

$$\boldsymbol{y}_{i,j} = (a_1 + i\Delta y_1, a_2 + j\Delta y_2),$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

where $\Delta y_1 = (b_1 - a_1)/M_1$ and $\Delta y_2 = (b_2 - a_2)/M_2$. We also assume that $\tau_k = M_k/N_k$ is an integer greater than one for $k = 1, 2$.

Following section 2, each training and test point is assigned a class labeled 0 or 1 and is denoted, respectively, by χ_{ij} for every $0 \le i \le N_1, 0 \le j \le N_2$ and μ_{ij} for each $0 \le i \le M_1, 0 \le j \le M_2$. Given a kernel function $\ker : Z \times Z \to \mathbb{R}$, we can define the kernel matrix

$$H_{\ker} = (\ker(\boldsymbol{x}_{i,j}, \boldsymbol{x}_{k,l}))_{0 \le i,k \le N_1, 0 \le j,l \le N_2} \in \mathbb{R}^{N_1 N_2 \times N_1 N_2},$$

with the indices arranged suitably.

We note first that the NTK must necessarily outperform the CK on the training points and that both must be superior to the trained NN, i.e.,

$$\ell_{0,\mathrm{NTK}}\left(\boldsymbol{\alpha}^{*,\mathrm{NTK}}\right) \le \ell_{0,\mathrm{CK}}\left(\boldsymbol{\alpha}^{*,\mathrm{CK}}\right) \le \ell_{0,\mathrm{NN}}\left(\theta\right). \tag{5.1}$$

The latter follows from the fact that the NN can viewed as a suboptimal CK approximator in the manner of Remark 3.2. The former is most easily seen by rewriting (5.1) in terms of the feature maps: since

$$\ell_{0,\mathrm{CK}}\left(\boldsymbol{\alpha}^{*,\mathrm{CK}}\right) = \widetilde{\ell}_{0,\mathrm{CK}}\left(\boldsymbol{\beta}^{*,\mathrm{CK}}\right),$$

from (2.18), and $\Phi_{\mathrm{NTK}} = (\Phi_{\mathrm{CK}} \;\; \Phi_{\mathrm{E}})^{\top}$, setting $\boldsymbol{\gamma} = (\boldsymbol{\beta}^{*,\mathrm{CK}} \;\; \mathbf{0})^{\top} \in \mathbb{R}^{|\theta|}$ yields

$$\ell_{0,\mathrm{CK}}\left(\boldsymbol{\alpha}^{*,\mathrm{CK}}\right) = \widetilde{\ell}_{0,\mathrm{CK}}\left(\boldsymbol{\beta}^{*,\mathrm{CK}}\right) = \widetilde{\ell}_{0,\mathrm{NTK}}\left(\boldsymbol{\gamma}\right) \ge \widetilde{\ell}_{0,\mathrm{NTK}}\left(\boldsymbol{\beta}^{*,\mathrm{NTK}}\right) = \ell_{0,\mathrm{NTK}}\left(\boldsymbol{\alpha}^{*,\mathrm{NTK}}\right). \tag{5.2}$$

In order to show that an equivalence result in the manner of Theorem 4.1 holds for the test errors, we follow a strategy similar to that for the function regression problem in section 4:

(a) identify two cases that permit a transfer of information from training to test nodes;
(b) extend (5.1) to an equivalence result on the training nodes for the two kernels

5.2. Another pair of lemmas. We begin by noting that

$$\ell_{0,\ker}(\boldsymbol{\alpha}^{*,\ker}) \le \ell_{1,\ker}(\boldsymbol{\alpha}^{*,\ker}), \qquad \ell_{0,\mathrm{NN}}(\theta) \le \ell_{1,\mathrm{NN}}(\theta), \tag{5.3}$$

simply by virtue of the fact that every training point is also a test point. To complete (a), we need to bound the error on the finer grid by that on the coarser grid. The quantity of interest is the linear combination of the feature map components

$$\psi_{\ker}(\boldsymbol{z}) = \sum_{m=1}^{|\theta|} \Phi_{\ker,m}(\boldsymbol{z}) \boldsymbol{\beta}_m^{*,\ker}, \tag{5.4}$$

as it determines the size of the gap between $\boldsymbol{z} \in Z$ and the separating hyperplane in the feature space. However, we also need to keep track of the class label that should be associated with $\boldsymbol{z}$, as that determines how it shows up in the loss function (2.17). Let $\eta : Z \to \{0, 1\}$ be the true class label for every point (so that, e.g., $\chi_{ij} = \eta(\boldsymbol{x}_{i,j})$ and $\mu_{ij} = \eta(\boldsymbol{y}_{i,j})$), and set

$$\widehat{\psi}_{\ker}(\boldsymbol{z}) = (1 - 2\eta(\boldsymbol{z}))\, \psi_{\ker}(\boldsymbol{z}). \tag{5.5}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Using this function, we can write the minimized training and test losses as simply

$$\widetilde{\ell}_{0,\mathrm{ker}} = \sum_{i=0}^{N_1}\sum_{j=0}^{N_2} \ln\left[1+\exp\left(\widehat{\psi}_{\mathrm{ker}}(\boldsymbol{x}_{i,j})\right)\right] \tag{5.6}$$

and

$$\widetilde{\ell}_{1,\mathrm{ker}} = \sum_{i=0}^{M_1}\sum_{j=0}^{M_2} \ln\left[1+\exp\left(\widehat{\psi}_{\mathrm{ker}}(\boldsymbol{y}_{i,j})\right)\right], \tag{5.7}$$

where we have suppressed the dependence on $\boldsymbol{\beta}^{*,\mathrm{ker}}$ for clarity of exposition. For any $0 \le i \le N_1 - 1$ and $0 \le j \le N_2 - 1$, we also define

$$\Lambda_{ij} = \{\boldsymbol{x}_{i,j}, \boldsymbol{x}_{i+1,j}, \boldsymbol{x}_{i,j+1}, \boldsymbol{x}_{i+1,j+1}\}$$

to make it easier to navigate the training grid.

The two regimes that interest us are a corner maximum condition and Lipschitz continuity. We make these precise in the following lemmas: the proofs are provided in section SM2 of the supplementary materials.

LEMMA 5.1. *Suppose that on every rectangle with corners Λ_{ij}, the function $\widehat{\psi}_{ker}$ achieves its maximum value on Λ_{ij}. Then*

$$\widetilde{\ell}_{1,ker} \le \tau_1\tau_2\widetilde{\ell}_{0,ker}, \tag{5.8}$$

We say that the labeling function η possesses the *matching property* with respect to the given training and test nodes if every test point $\boldsymbol{y}_{k,l}$ is contained in an enclosing rectangle with corners $\Lambda_{k'l'}$ such that η agrees with $\eta(\boldsymbol{y}_{k,l})$ on at least one corner point. In other words, we do not have the matching property if and only if there exists some test point $\boldsymbol{y}_{k,l}$ such that, for any rectangle with corners $\Lambda_{k'l'}$ that may enclose it, we have $\eta(\boldsymbol{y}_{k,l}) \notin \eta(\Lambda_{k'l'})$.

LEMMA 5.2. *Suppose that ψ_{ker} is Lipschitz continuous on D, with Lipschitz constant L_{ker}, and η possesses the matching property. Then*

$$\widetilde{\ell}_{1,ker} \le \exp(hL_{ker})\tau_1\tau_2\widetilde{\ell}_{0,ker}. \tag{5.9}$$

In combination with (5.3), Lemmas 5.1 and 5.2 allow us to move seamlessly between training and test losses. Note that the assumptions underlying these lemmas can only conceivably be satisfied by smooth ψ_{ker}, thus almost completely ruling out the suitability of ReLU activations. On the other hand, the Tanh activation function equips the kernels with Lipschitz continuity by default; the matching property is a technical assumption to rule out pathological cases where the label of a test point disagrees with those of all of its neighbouing training points.

5.3. Performance comparison. Next, we address task (b), outlined at the end of subsection 5.1. Note that for any $0 \le i \le N_1$, $0 \le j \le N_2$, we have

$$\begin{aligned}
&\ln\left[1+\exp\left(\widehat{\psi}_{\mathrm{CK}}\left(\boldsymbol{x}_{i,j}\right)\right)\right] \\
&\quad= \ln\left[1+\exp\left(\widehat{\psi}_{\mathrm{CK}}\left(\boldsymbol{x}_{i,j}\right)-\widehat{\psi}_{\mathrm{NTK}}\left(\boldsymbol{x}_{i,j}\right)\right)\exp\left(\widehat{\psi}_{\mathrm{NTK}}\left(\boldsymbol{x}_{i,j}\right)\right)\right] \\
&\quad\le \max\left\{1,\exp\left(\widehat{\psi}_{\mathrm{CK}}\left(\boldsymbol{x}_{i,j}\right)-\widehat{\psi}_{\mathrm{NTK}}\left(\boldsymbol{x}_{i,j}\right)\right)\right\}\ln\left[1+\exp\left(\widehat{\psi}_{\mathrm{NTK}}\left(\boldsymbol{x}_{i,j}\right)\right)\right],
\end{aligned}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

where we used the inequality

$$\ln(1+\rho x) \le \rho \ln(1+x) \tag{5.10}$$

for $\rho \ge 1$ and $x \ge 0$. Set

$$\omega = \max_{0\le i\le N_1, 0\le j\le N_2} \exp\left(\widehat{\psi}_{\mathrm{CK}}(\boldsymbol{x}_{i,j}) - \widehat{\psi}_{\mathrm{NTK}}(\boldsymbol{x}_{i,j})\right); \tag{5.11}$$

it follows from (5.1) that $\omega \ge 1$. We can therefore write

$$\widetilde{\ell}_{0,\mathrm{CK}} \le \omega \widetilde{\ell}_{0,\mathrm{NTK}}. \tag{5.12}$$

Combining (5.1) and (5.12) with (5.3) and Lemmas 5.1 and 5.2 then yields the following two results.

THEOREM 5.1. *Suppose that on every rectangle with corners Λ_{ij} the function $\widehat{\psi}_{ker}$, where ker refers to the NTK or the CK, achieves its maximum values on Λ_{ij}. Then*

$$\ell_{1,ker} \le (\tau_1\tau_2)\ell_{1,NN}. \tag{5.13}$$

Moreover, if on every rectangle with corners Λ_{ij} the functions $\widehat{\psi}_{NTK}$ and $\widehat{\psi}_{CK}$ achieve their maximum values on Λ_{ij}, then there exist constants $C_1, C_2 \ge 1$ such that

$$C_1^{-1}\ell_{1,NTK} \le \ell_{1,CK} \le C_2\ell_{1,NTK}. \tag{5.14}$$

Proof. We have

$$\begin{aligned} \ell_{1,\mathrm{ker}} &\le (\tau_1\tau_2)\ell_{0,\mathrm{ker}} && (\because \text{Lemma 5.1}) \\ &\le (\tau_1\tau_2)\ell_{0,\mathrm{NN}} && (\because (5.1)) \\ &\le (\tau_1\tau_2)\ell_{1,\mathrm{NN}} && (\because (5.3)). \end{aligned} \tag{5.15}$$

For the second part, the first inequality in (5.14) follows from an identical argument and setting $C_1 = \tau_1\tau_2$. For the second inequality, observe that

$$\begin{aligned} \ell_{1,\mathrm{CK}} &\le (\tau_1\tau_2)\ell_{0,\mathrm{CK}} && (\because \text{Lemma 5.1}) \\ &\le (\tau_1\tau_2\omega)\ell_{0,\mathrm{NTK}} && (\because (5.12)) \\ &\le (\tau_1\tau_2\omega)\ell_{1,\mathrm{NTK}} && (\because (5.3)), \end{aligned} \tag{5.16}$$

so $C_2 = \tau_1\tau_2\omega$ yields the desired result. ∎

THEOREM 5.2. *Suppose that η possesses the matching property and ψ_{ker}, where ker refers to the NTK or the CK, is Lipschitz continuous on D with Lipschitz constant L_{ker}. Then*

$$\ell_{1,ker} \le e^{hL_{ker}}\ell_{1,NN}. \tag{5.17}$$

In case both ψ_{NTK} and ψ_{CK} are Lipschitz continuous on D, then there exist constants $D_1, D_2 \ge 1$ such that

$$D_1^{-1}\ell_{1,NTK} \le \ell_{1,CK} \le D_2\ell_{1,NTK}. \tag{5.18}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Proof. We have

$$\begin{aligned} \ell_{1,\mathrm{ker}} &\le e^{hL_{\mathrm{ker}}}\ell_{0,\mathrm{ker}} && (\because \text{Lemma 5.2}) \\ &\le e^{hL_{\mathrm{ker}}}\ell_{0,\mathrm{NN}} && (\because (5.1)) \\ &\le e^{hL_{\mathrm{ker}}}\ell_{1,\mathrm{NN}} && (\because (5.3)). \end{aligned} \tag{5.19}$$

The first inequality in (5.18) follows from a similar argument and setting $D_1 = e^{hL_{\mathrm{NTK}}}$. For the second inequality, we have

$$\begin{aligned} \ell_{1,\mathrm{CK}} &\le e^{hL_{\mathrm{CK}}}\ell_{0,\mathrm{CK}} && (\because \text{Lemma 5.2}) \\ &\le e^{hL_{\mathrm{CK}}}\ell_{0,\mathrm{NTK}} && (\because (5.12)) \\ &\le \omega e^{hL_{\mathrm{CK}}}\ell_{1,\mathrm{NTK}} && (\because (5.3)). \end{aligned} \tag{5.20}$$

Setting $D_2 = \omega e^{hL_{\mathrm{CK}}}$ yields (5.18). ∎

6. Numerical tests. In this section, we present numerical evidence to support the theoretical findings established in sections 4 and 5. In addition, we identify myriad benefits of employing CK approximations over those coming from NN and NTK, including better conditioning, inexpensive gains in accuracy, and improved robustness.

6.1. Illustrative examples for regression and classification. We first consider the approximation problem for smooth functions $f : [-1, 1] \to \mathbb{R}$. As examples, we use

$$f_1(x) = e^{\sin(2\pi x)}, \qquad f_2(x) = e^{3x}, \qquad f_3(x) = \cos\left(e^{3x}\right). \tag{6.1}$$

We note that f_3 oscillates rapidly close to $x = 1$ so it poses a particularly stiff challenge to NN and kernel approximators alike.

The sizes of the training and testing grids, as defined in subsection 4.1, are fixed at $N = 200$ and $M = 600$, respectively, so $\tau = 3$. We train NNs of the form (2.1) with $L = 3$ and $d_l = d = 128$ for $1 \le l \le 3$ for 2400 epochs using the loss function defined in (2.2) and the weights given in (4.1). We employ the ADAM optimizer with the learning rate set at 10^{-3} and primarily use Tanh as the activation function; we explore other choices later.

After training an NN, we assemble the CK and NTK and compute the corresponding kernel approximations (2.8). The details of the algorithm used for extracting the NTK are given in section SM3 of the supplementary materials. The CK is easily obtained from the values of the last hidden layer and using (3.5). Since the feature space dimension for the CK is $(d+1)$, it allows for the use of the explicit feature map using (3.4), the results of which are indicated by CKJ (for "CK Jacobian") in the following plots. Specifically, we can assemble the corresponding Jacobian matrix at the training points and use its singular value decomposition (SVD) to minimize the loss (2.11). This has the advantage of not squaring the singular values, as happens when the kernel matrix is formed, and leads to lower errors due to better conditioning. We stress that this property is not afforded by the NTK, whose feature space dimension is generally prohibitively high.

In Figure 1, we show the test errors for the example target functions for 100 runs of the procedure described above. Owing to randomness in the initialization, the results vary over iterations; nevertheless, a clear separation is evident between the NN test errors and those of the two kernel approximations. More importantly, we note

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

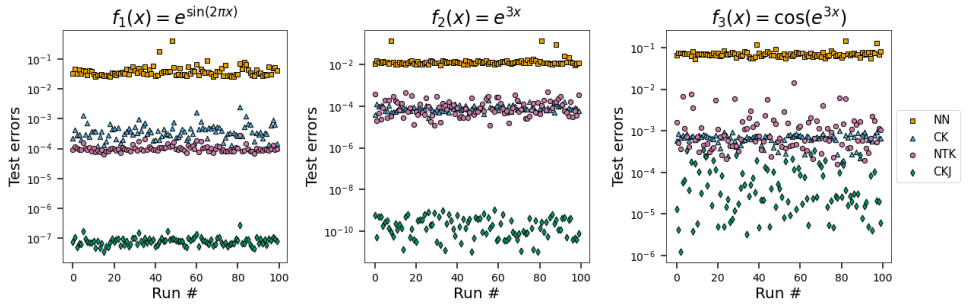


FIG. 1. *The results of function regression for* 100 *different NNs, trained for* 2,400 *epochs, and the corresponding approximations using the NTK, CK, and CKJ extracted from the NN at the end of the training.*

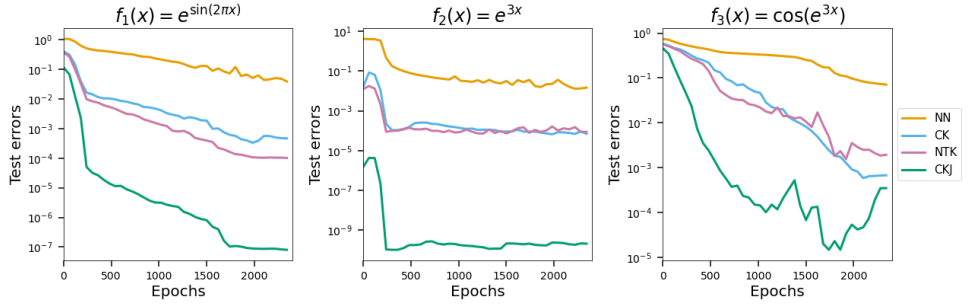


FIG. 2. *The test errors for NNs over the course of being trained to approximate the given functions and for the corresponding NTK, CK, and CKJ approximations. The errors are averaged over* 10 *iterations to reduce the effects of random initialization.*

that the latter pair are fairly similar so that, while one may appear to be superior over the other for a particular example (e.g., the NTK for f_1), the difference is only marginal compared to the supremacy both enjoy over the NN.

Figure 1 also shows that the CKJ test errors are an improvement on the kernel approximations by several orders of magnitude. This is a consequence of the much better conditioning possessed by the former: assembling the kernel matrix $\widehat{H}_{\text{ker}}$ squares the singular values and applying its pseudoinverse in (2.8) inflates round-off errors more than using the Jacobian does, with the result that the test errors plateau earlier. We emphasize that the use of the Jacobian is only possible for the CK since the dimension of the feature space corresponding to the NTK is prohibitively large.

These claims are further supported by the evolution of the errors over the course of the training, shown in Figure 2. The test errors are computed by averaging over 10 iterations to reduce the effect of random initialization. While the NN errors can be seen to decay gradually, the kernel approximations do so more rapidly, particularly over the first few hundred epochs. The close resemblance between the two error profiles is noteworthy and supports our contention that the CK can serve as a proxy for the NTK. Finally, we note that, as expected, the CKJ errors decay even faster and level off at a threshold several orders of magnitude lower.

In Figure 3, we show that these conclusions are also valid when we repeat the experiments with the width of the hidden layers in the NN set to $d = 256$. In particular,

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

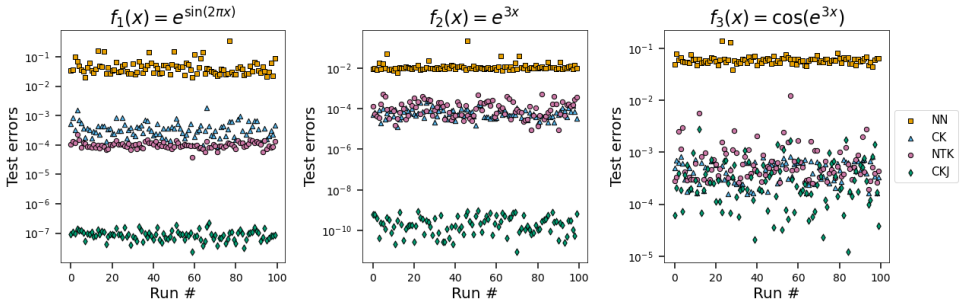


FIG. 3. *Function regression results for* 100 *NNs with the widths of the hidden layers set to* 256 *and the corresponding NTK, CK, and CKJ approximations.*

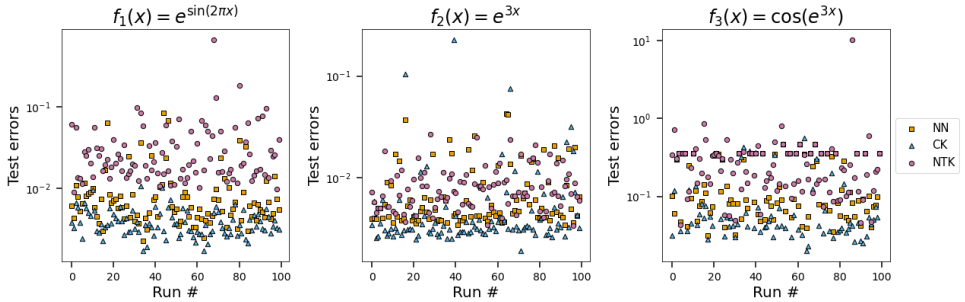


FIG. 4. *The test errors for function regression for* 100 *trained NNs using the ReLU activation function and the corresponding NTK and CK approximations.*

this assuages any concerns that the beneficial properties enjoyed by the CK and CKJ approximations may be a consequence of solving an overdetermined least-squares problem: with the larger width, this is no longer the case, and the similarities in the corresponding error plots in Figures 1 and 3 demonstrate that our assertions are independent of NN width.

The same, however, cannot be said about the activation functions. Figure 4 shows that using ReLU activations instead of Tanh leads to the erasure of the error segregation seen in the earlier diagrams. In addition, the NTK approximators paradoxically appear to perform the worst, with the CK results only slightly bettering the NN ones. Figure 5 highlights that this is generally the case over the course of the NN training as well. These observations can be explained as a combined effect of the overparameterization provided by the NTK and $\mathcal{S}_{\mathrm{NTK}}$ containing discontinuous functions (due to the derivatives of the ReLU activations; see section SM3 of the supplementary materials for more details). The resulting approximator manages to fit the training data very well but struggles with generalizing to the test data-points. In contrast, the $\mathcal{S}_{\mathrm{CK}}$ consists only of continuous (albeit nondifferentiable) functions, with the result that, while the approximation accuracy is lower than we observed with Tanh, it does not suffer from overfitting. This can be regarded as indicative of the dangers associated with overparameterization, the care that must be taken with ReLU-based approximators on unseen data, and the robustness possessed by the CK.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

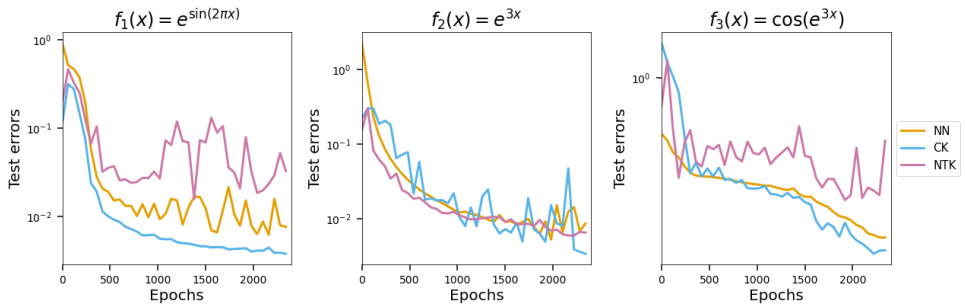


FIG. 5. *Averaged test errors for function regression over the course of training* 10 *NNs using ReLU activations and the corresponding NTK and CK approximations.*

Next, we present the results for binary logistic regression on $Z = [-1,1]^2$. We set $N_1 = 11$, $N_2 = 7$, $M_1 = 22$, and $M_2 = 21$ (so $\tau_1 = 2$ and $\tau_2 = 3$) to specify the uniform training and testing grids (see subsection 5.1 for the details). The labels are generated with respect to a known separating boundary $F(x_1, x_2) = 0$ for some $F : Z \to \mathbb{R}$, so the labeling function is $\eta(x_1, x_2) = (\text{sign}(F(x_1, x_2)) + 1)/2$; we consider two examples:

(6.2)
$$F_1(x_1, x_2) = 4x_1^2 - 3x_1 + 5x_2 - 1, \quad F_2(x_1, x_2) = 2x_1^3 - 0.6x_1^2 - 1.94x_1 + x_2 + 0.2.$$

We train NNs of the form (2.1), with $L = 2$ and $d_l = d = 128$ for $l = 1, 2$, until the training accuracy (i.e., the proportion of correctly identified points) crosses 0.85, or 4000 epochs have elapsed. We again employ the ADAM optimizer, with learning rate 10^{-5}, and use Tanh as the primary activation function. The NTK and CK are extracted as detailed before, and the respective training cross-entropies, defined in (2.9), are minimized by using Newton's method to solve $\nabla_{\boldsymbol{\alpha}} \ell_{0,\text{ker}}(\boldsymbol{\alpha}) = \mathbf{0}$. We consider both cross-entropy loss values and classification accuracies as assessment metrics on the test points.

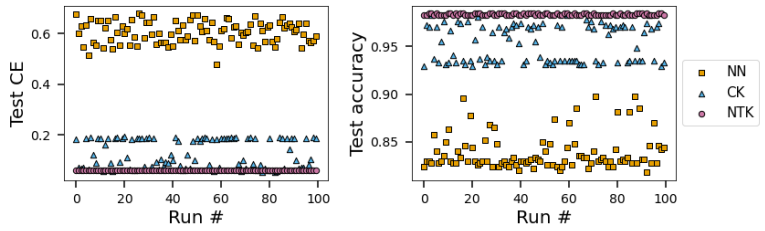
The plots in Figure 6 show the results of 100 iterations of this procedure. A clear partition is visible in the performance metrics of the NN and the two kernels on the test datasets. At the same time, while the CK is generally outperformed by the NTK, the degree of improvement is dwarfed by the separation from the NN results.

Figure 7 shows that this is still the case, even when the hidden widths of the NN are halved to 64, thus demonstrating that the CK performance is not simply a consequence of overparameterization. However, Figure 8 shows that the clear gap in performance vanishes when the activation function employed is ReLU. As in the case of function regression, this is explainable by overfitting as a result of the volatile mix of overparameterization and discontinuous feature map components.

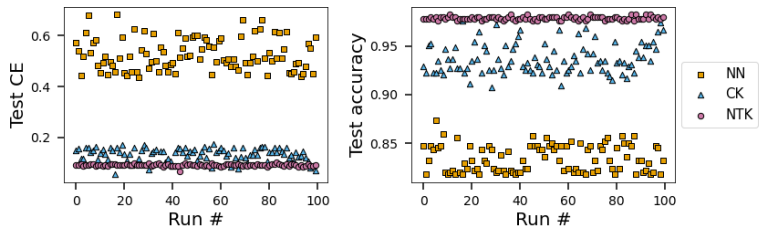
Finally, Figure 9 shows the evolution of the performance metrics over the course of the NN training. We again show the average of 10 iterations to reduce the effect of random initialization. The separation alluded to earlier is evident throughout, as is the relative proximity of the loss and accuracy values of the two kernels. Moreover, we note that the gaps shrink with training, so much so that the CK metrics are barely distinguishable from the NTK ones; the NN performance improves, but a sizeable gulf nevertheless persists. In particular, this highlights a crucial point: even when the two kernels have been extracted from a partially trained NN, they yield better results than the NN does at the end of training and when its metrics have plateaued.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

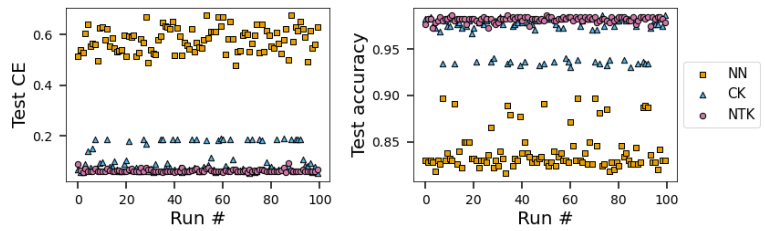


(a) $F_1(x_1, x_2) = 4x_1^2 - 3x_1 + 5x_2 - 1 = 0$

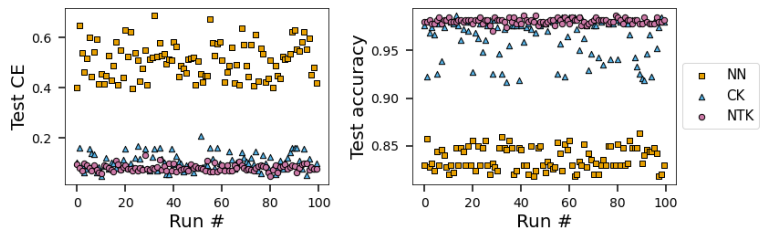


(b) $F_2(x_1, x_2) = 2x_1^3 - 0.6x_1^2 - 1.94x_1 + x_2 + 0.2 = 0$

FIG. 6. *The test cross-entropies and accuracies for logistic regression performed with* 100 *NNs and the corresponding NTK and CK results.*



(a) $F_1(x_1, x_2) = 4x_1^2 - 3x_1 + 5x_2 - 1 = 0$

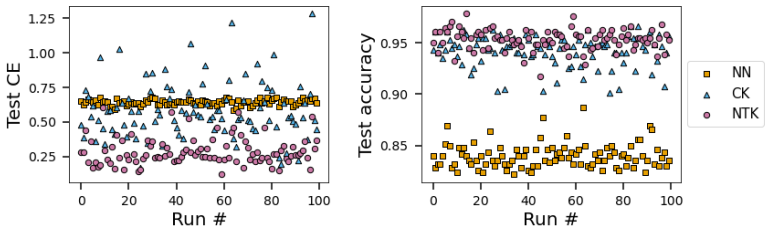


(b) $F_2(x_1, x_2) = 2x_1^3 - 0.6x_1^2 - 1.94x_1 + x_2 + 0.2 = 0$

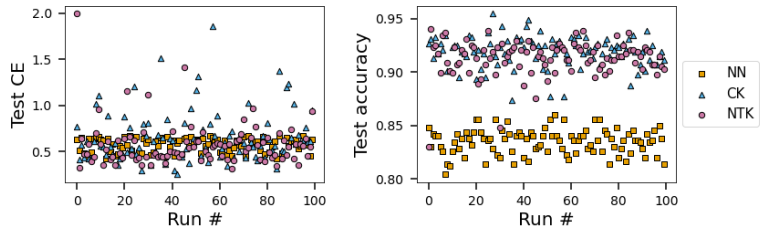
FIG. 7. *The cross-entropies and accuracies for the test dataset for logistic regression performed with* 100 *NNs with hidden layer widths set to* 64 *and the corresponding NTK and CK results.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

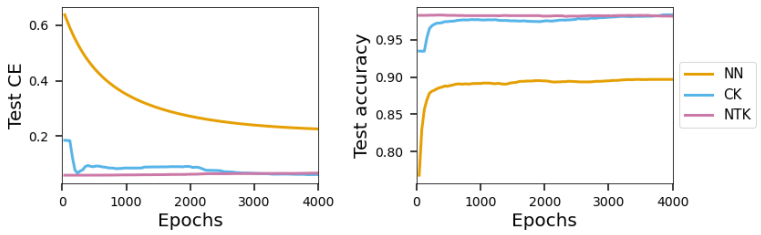


(a) $F_1(x_1, x_2) = 4x_1^2 - 3x_1 + 5x_2 - 1 = 0$

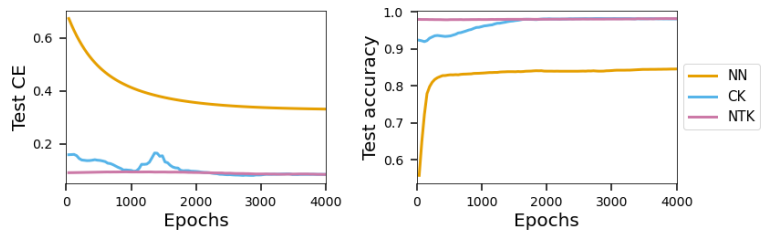


(b) $F_2(x_1, x_2) = 2x_1^3 - 0.6x_1^2 - 1.94x_1 + x_2 + 0.2 = 0$

FIG. 8. *Test results for* 100 *NNs using the ReLU activation function for logistic regression and the corresponding NTK and CK metrics.*



(a) $F_1(x_1, x_2) = 4x_1^2 - 3x_1 + 5x_2 - 1 = 0$



(b) $F_2(x_1, x_2) = 2x_1^3 - 0.6x_1^2 - 1.94x_1 + x_2 + 0.2 = 0$

FIG. 9. *Evolution of the test metrics, averaged over* 10 *iterations, of NNs used to solve the logistic regression problem, and the corresponding NTK and CK results.*

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

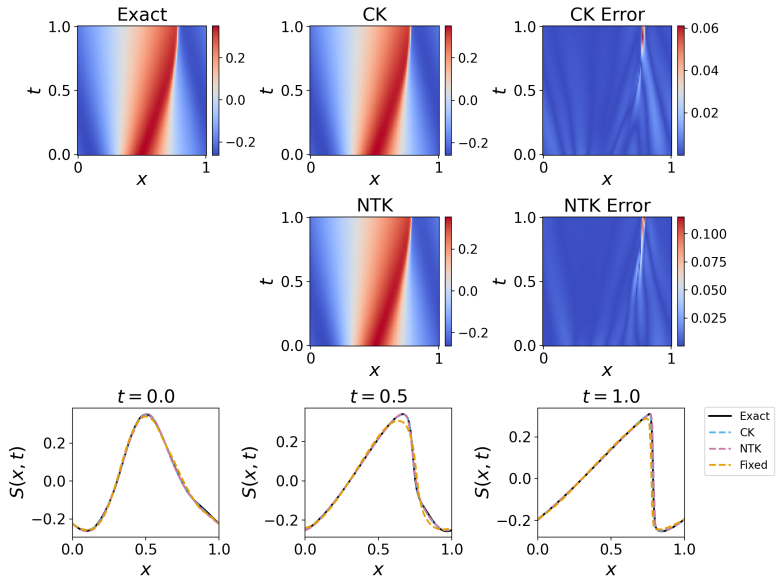


FIG. 10. *Results for viscous Burgers equation for* $\nu = 0.001$.

This underscores that using the CK (i.e., explicitly retraining only the last layer of the NN) is a low-cost recipe for significantly improving the accuracy of an NN.

6.2. Physics-informed operator learning. The viscous one-dimensional Burgers equation with periodic boundary conditions presented in [12, 13, 38, 39] presents a challenge for Deep Operator Networks (DeepONets) [24]. The system is given by

$$\frac{\partial s}{\partial t} + s\frac{\partial s}{\partial x} - \nu\frac{\partial^2 s}{\partial x^2} = 0,\ (x,t) \in [0,1] \times [0,1]. \tag{6.3}$$

$$s(x,0) = u(x),\ x \in [0,1], \tag{6.4}$$

$$s(0,t) = s(1,t),\ t \in [0,1], \tag{6.5}$$

$$\frac{\partial s}{\partial x}(0,t) = \frac{\partial s}{\partial x}(1,t),\ t \in [0,1], \tag{6.6}$$

where ν is the viscosity. To provide a direct comparison, we run samples with hyperparameters as close as possible to [39]. We generate initial conditions $u(x)$ from a Gaussian random field $\sim \mathcal{N}(0, 25^2(-\Delta + 5^2 I)^{-4})$, and the initial conditions are sampled at $P_{IC} = 101$ uniformly spaced locations on $x \in [0,1]$, and the boundary conditions are randomly sampled at $P_{BC} = 100$ locations on $(x,t) = (0,t)$ and $(x,t) = (1,t)$. The residual is evaluated on $P_p = 2,500$ randomly sampled collocation points from the interior of the domain. The performance is tested by generating an additional $N_{test} = 500$ initial conditions $u(x)$ and simulating the solution with MATLAB using the Chebfun [6] package following the method in [39]. We train with $N = 1000$ samples of the initial condition $u(x)$.

In [39], the use of the NTK to assign weights dynamically (for each point included in the loss function and for each training iteration) in front of the various terms in the loss function used to train the DeepONet was shown to perform better than using fixed predetermined weights. However, the calculation of the NTK can be expensive. Alternatively, we can use the CK to assign the weights in the loss function. Results comparing the CK and the NTK and using predetermined fixed weights are shown in Figure 10 and Table 1. The use of the CK not only leads to an improvement of the

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 1

Comparison of the results and computational time for the viscous Burgers equation for $\nu = 0.001$. *The reported error is the relative* ℓ_2 *error over* 500 *randomly sampled initial conditions not seen in training. The results use local NTK and CK weights, as defined in* [39] *for the NTK. The runtime is calculated using one node of an NVIDIA* A100 *GPU.*

Weights	Fixed	NTK	CK
Error	$9.178\% \pm 8.027\%$	$6.301\% \pm 6.876\%$	$3.079\% \pm 4.375\%$
Time (h)	1.576	6.051	2.288

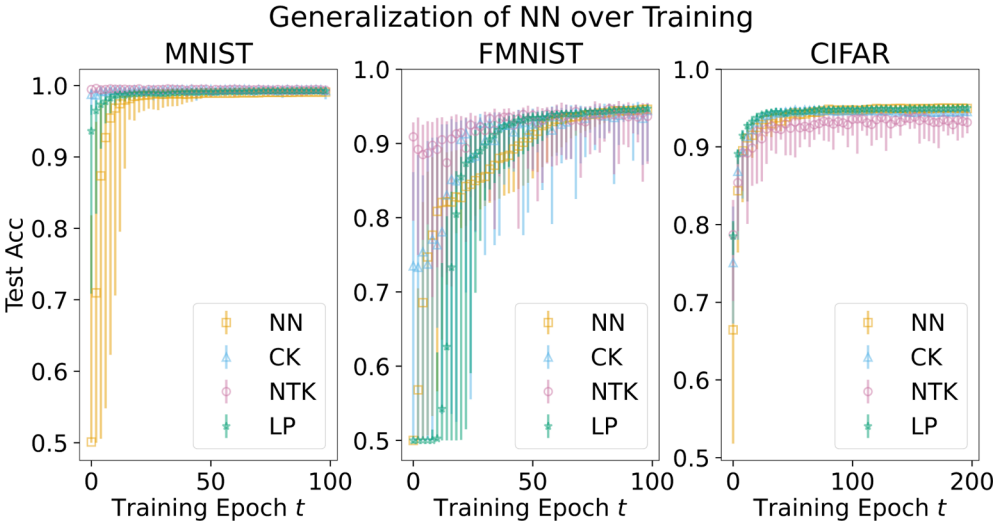


FIG. 11. *A small four layer CNN is trained to classify the first two classes of the MNIST, FMNIST, and CIFAR datasets. After every epoch, we freeze all weights except for the linear regression head and train the head for an additional* 5 *epochs (LP). We calculate the* CK *and* NTK*, train the corresponding kernel logistic regression models, and compare both against the NN for generalization performance on the held out test data. We do this* 33 *times with different random seeds: uncertainty bars show the* 1*-STD equivalent quantile. We observe across each dataset that after an initial feature learning phase, the NTK and CK kernel models become generalizable* faster than *the neural network.*

test accuracy compared to the NTK but also cuts the training time by around 62% of the training time needed in the case of the NTK.

6.3. Computer vision classification. A small four layer CNN is trained to classify the first two classes of the MNIST, FMNIST, and CIFAR datasets. At regular intervals in training, the weights are stored and used to calculate the CK and NTK for the train and test datasets offline. Kernel logistic regression models are then trained on both the CK and NTK using the same train-test split as the neural networks. We also perform linear probing (LP) by freezing all the weights at each epoch, except for the linear regression head, and training the head for another 5 epochs. We repeat this experiment 33 times with different seeds to evaluate the distribution of trained models, CKs, and NTKs. In Figure 11, we report the performance of the NN, NTK logistic regression model, and the CK logistic regression model over the history of training. Similar to [9], we find that the NTK logistic regression model quickly saturates to nearly the same final performance of the NN classifier. We also observe that the CK logistic regression model also becomes performant earlier in training than the NN model but only marginally later than the NTK model. Combined with the significant computational savings accruing from the calculation of the CK compared to the NTK,

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 2

Performance metrics of various training methods for Falcon 7B, Llama-2 7B, and GPT-2 on classification of Sentiment140's manually curated data. Note that LP in this table corresponds to only training the logistic regression layer without any fine-tuning of the model.

Model	Method	Train acc.	Test acc.	Time/epoch (s)	GPU memory (GB)
Falcon 7B	FT	$99.95 \pm 0.02\%$	$91.8 \pm 0.9\%$	103.9	56.5
	LP	$96.4 \pm 0.12\%$	$88.9 \pm 0.8\%$	8.8	14.3
Llama-2 7B	FT	$99.81 \pm 0.05\%$	$90.5 \pm 0.75\%$	105.27	53.5
	LP	$97.17 \pm 0.12\%$	$87.4 \pm 0.57\%$	9.49	13.8
GPT-2	FT	$99.84 \pm 0.03\%$	$86.67\% \pm 0.94\%$	12.72	3.7
	LP	$93.18 \pm 0.24\%$	$85.91 \pm 0.94\%$	3.66	1.87

this underscores the advantages of complementing (partial or full) NN training with a CK machine.

7. Application to foundation models. In this section, we demonstrate the effectiveness of our prescribed training strategy on foundation models. Specifically, we make use of the pretrained large language models (LLMs) GPT-2 [31], Falcon 7B [29], and Llama-2 7B [35] for sentiment classification on a human-annotated subset of the Sentiment140 dataset consisting of 359 tweets, all of which are labeled either positive or negative based on the expressed sentiment [10], rather than the entire Sentiment140 dataset due to a distribution shift between the train and test split as reported in [36]. We evaluate two approaches:

(FT) We fine-tune by starting from pretrained weights, attaching a randomly initialized classification head and train for 10 epochs, allowing *all* the weights to update.

(LP) We perform linear probing by extracting a feature vector at the final hidden layer and training a logistic regression classifier for 10 epochs.

We note that these procedures can be seen as analogues of the NN and CK training, respectively, studied in the earlier sections. In particular, both NN training and FT entail updates to all the parameters in the architecture, while the CK and LP approaches employ the feature map from the last hidden layer of a partially-trained architecture for logistic regression. Consequently, while the latter approach is significantly less expensive than the former, it can, as shown in the earlier sections, yield comparable (when LP substitutes for FT) or even a marked improvement in accuracy (when LP is combined with FT).

We use pretrained weights from Hugging Face for the transformer layers [40]. Furthermore, to establish a baseline, we also perform linear probing with the LLMs initialized with random weights in the final classification head drawn from the truncated normal distribution $\mathcal{N}(0, 0.02)$ [30, 31]. All experiments are performed 30 times with the training and testing data shuffled. The averaged results, along with the standard errors, are reported in Table 2 with further details about the hyperparameters provided in Table SM1 of the supplementary materials. We note that exclusively training the classification head (LP) on the in situ feature representation of the pretrained model attains at least 96% of the generalization performance of updating all of the weights (FT) for all three LLMs at a fraction of the computational and memory costs.

To further highlight the advantages of short training time and low memory cost of performing an LP, we investigate the effect of quickly training a linear probe after any given amount of FT training epochs (including the case of no FT given at epoch 0).

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

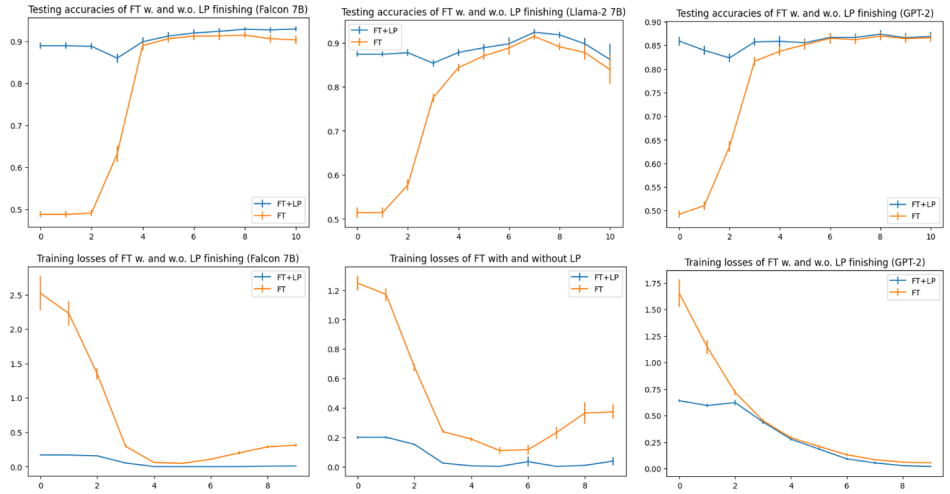


FIG. 12. *Test accuracy of training LLMs over* 10 *epochs. After each epoch, we freeze all weights except for the final layer and train the final layer for* 10 *additional epochs. We plot averages over* 30 *seeds, error bars indicating standard error of the mean. Note that LP in Table* 2 *corresponds to the leftmost point (epoch* 0*) while FT in Table* 2 *corresponds to the right most point (epoch* 10*) for the testing accuracies.*

Figure 12 shows the average test accuracies of FT training after each epoch, compared to the performance of the model if we had stopped FT training and instead trained 10 epochs with LP. In all cases, we observe that on average, topping off the FT procedure with some additional LP steps bounds the accuracies from above as well as the losses from below. These bounds on the test and training accuracies coupled with gains in wall clock performance as well as memory allocation (compared with FT) makes a strong argument for limiting the number of FT epochs in exchange for topping off with LP training.

8. Discussion. An optimally trained NN, in the infinite width limit, can be shown to converge to a kernel machine with a closed form known a priori [1, 20]. As explained in Remark 3.2, for finite width networks, a kernel machine relying on the empirical NTK would be a better approximation than the parent NN on the training data. The numerical tests in section 6 confirm that this is also the case on test datasets across different problems, network widths, and training stages, provided the Tanh activation function is used. However, evaluating the resulting approximations (e.g., (2.8) and (2.10)) at a new point requires computing the NTK by pairing this point with every training point, which is a highly expensive undertaking for networks used in practical applications. Worse still, the NTK performance may be inferior to that of the NN when ReLU activations are employed (e.g., Figures 4 and 8) due to discontinuities in its feature map components. Furthermore, the resulting sensitivity to small perturbations in the inputs may also leave these approximations singularly susceptible to adversarial attacks.

In this paper, we studied the properties of approximations relying on the CK and found, both theoretically and empirically, that their performance is closely tied to the NTK results, provided certain regularity conditions on the approximations are met. These are satisfied by default for the Tanh activation function, thus establishing that the gains in accuracy for the NTK over the NN are also inherited by the CK approximations. Importantly, the training cost of a CK machine is significantly lower than for continuing to train the NN, yielding a closed-form formula for function regression

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

(2.8) and a convex optimization problem for logistic regression (2.9). In addition, its evaluation on new data-points is as inexpensive as evaluating the NN on the same points. As a result, the CK achieves a boost in accuracy over the NN comparable to that of the NTK while being strikingly cheaper to use than the latter and much easier to train than the former.

The CK also possesses features that, in some settings, make it even more favorable than the NTK. Due to the relatively low dimension of its feature space, the usage of its feature map form (2.12) for function regression is computationally viable. This approximation is much better conditioned than the corresponding kernel form, with the result that round-off errors are not magnified excessively and the approximation errors can achieve a much lower plateau, as shown in Figures 1, 2, and 3. In addition, the components of the feature map corresponding to the CK are continuous, even when using ReLU activations. As a consequence, the resulting approximations are less prone to overfitting and more robust to noise in the inputs.

We note that training a CK machine is equivalent to finding the optimal weights to attach to the neurons in the last hidden layer. Our work then suggests that a low-cost strategy for improving the accuracy of a trained NN is to simply retrain the last layer of parameters. This recipe does not alter the form of the neurons (i.e., the feature map components for the CK) but, as we have shown, can lead to significant enhancements in approximation accuracy. The regularity of the activation function is a key determinant in the success of this strategy. Observe that the choice of the activation function does not appear to affect the NN approximation errors too much, but the kernel performance is closely tied to it (cf. Figures 1 and 4, and Figures 6 and 8). This indicates that the effects of the regularity of the activation function are realized only on the kernel machines and that the NN is largely agnostic to it.

In addition to simple illustrative examples, we have tested the performance of CK compared to the NTK also for more elaborate cases. First, we used the CK to replace the NTK as a way of assigning weights for physics-informed operator learning (a regression task). Our numerical results show that the CK can lead to comparable performance to the NTK while significantly reducing the training time. Second, we used the CK and NTK to train surrogates for logistic regression for vision classification (MNIST, FMNIST, and CIFAR). We show that the NTK and CK both become performant earlier in training compared to the NN classifier. Even though the CK becomes performant more slowly than the NTK, the computational efficiency of CK compared to NTK makes a case for its use.

The use of our strategy on foundation models for classification illustrates that freezing the feature map and training only the last layer of parameters yields performance that compares favorably with full architecture training. In addition, this approach might even lend improved robustness to the predictive capabilities in that the limited degrees of freedom resist overfitting on the training data. Our results therefore hint at the possibility that early stopping for model fitting might be beneficial in more ways than is currently surmised and that there might be subtle procedures to modulate stopping times across various parts of the network. Our recommended approach offers a computational speed up without the large trade-off in accuracy seen in other efficient fine-tuning methods.

REFERENCES

[1] S. Arora, S. S. Du, W. Hu, Z. Li, R. R. Salakhutdinov, and R. Wang, *On exact computation with an infinitely wide neural net*, Adv. Neural Inform. Process. Syst., 32 (2019).

[2] P. L. Bartlett, A. Montanari, and A. Rakhlin, *Deep learning: A statistical viewpoint*, Acta Numer., 30 (2021), pp. 87–201, https://doi.org/10.1017/S0962492921000027.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[3] M. Belkin, *Fit without fear: Remarkable mathematical phenomena of deep learning through the prism of interpolation*, Acta Numer., 30 (2021), pp. 203–248, https://doi.org/10.1017/S0962492921000039.

[4] B. E. Boser, I. M. Guyon, and V. N. Vapnik, *A training algorithm for optimal margin classifiers*, in Proceedings of the Fifth Annual Workshop on Computational Learning Theory, 1992, pp. 144–152.

[5] I. Daubechies, R. DeVore, S. Foucart, B. Hanin, and G. Petrova, *Nonlinear approximation and (deep) ReLU networks*, Constr. Approx., 55 (2022), pp. 127–172, https://doi.org/10.1007/s00365-021-09548-z.

[6] T. A. Driscoll, N. Hale, and L. N. Trefethen, *Chebfun Guide*, Pafnuty Publications, 2014, http://www.chebfun.org/docs/guide/.

[7] A. W. Engel, Z. Wang, N. Frank, I. Dumitriu, S. Choudhury, A. Sarwate, and T. Chiang, *Faithful and efficient explanations for neural networks via neural tangent kernel surrogate models*, in Proceedings of the Twelfth International Conference on Learning Representations, https://openreview.net/forum?id=yKksu38BpM, 2024.

[8] Z. Fan and Z. Wang, *Spectra of the conjugate kernel and neural tangent kernel for linear-width neural networks*, Adv. Neural Inform. Process. Syst., 33 (2020), pp. 7710–7721.

[9] S. Fort, G. K. Dziugaite, M. Paul, S. Kharaghani, D. M. Roy, and S. Ganguli, *Deep learning versus kernel learning: An empirical study of loss landscape geometry and the time evolution of the neural tangent kernel*, Adv. Neural Inf. Process. Syst., 33 (2020), pp. 5850–5861.

[10] A. Go, R. Bhayani, and L. Huang, *Twitter sentiment classification using distant supervision*, in Stanford CS224N Project Report, 2009, https://api.semanticscholar.org/CorpusID:18635269.

[11] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[12] A. A. Howard, S. H. Murphy, S. E. Ahmed, and P. Stinis, *Stacked Networks Improve Physics-Informed Training: Applications to Neural Networks and Deep Operator Networks*, preprint, arXiv:2311.06483, 2023.

[13] A. A. Howard, M. Perego, G. E. Karniadakis, and P. Stinis, *Multifidelity deep operator networks for data-driven and physics-informed problems*, J. Comput. Phys., 493 (2023), 112462, https://doi.org/10.1016/j.jcp.2023.112462.

[14] A. Jacot, F. Gabriel, and C. Hongler, *Neural tangent kernel: Convergence and generalization in neural networks*, Adv. Neural Inform. Process. Syst., 31 (2018).

[15] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, *Physics-informed machine learning*, Nature Rev. Phys., 3 (2021), pp. 422–440, https://doi.org/10.1038/s42254-021-00314-5.

[16] J. D. M.-W. C. Kenton and L. K. Toutanova, *Bert: Pre-training of deep bidirectional transformers for language understanding*, in Proceedings of naacL-HLT, Vol. 1, 2019, 2.

[17] Y. Kim, *Convolutional neural networks for sentence classification*, in Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2014.

[18] N. Kovachki, S. Lanthaler, and S. Mishra, *On universal approximation and error bounds for Fourier neural operators*, J. Mach. Learn. Res., 22 (2021), pp. 13237–13312.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, *Imagenet classification with deep convolutional neural networks*, Adv. Neural Inform. Process. Syst., 25 (2012).

[20] J. Lee, L. Xiao, S. Schoenholz, Y. Bahri, R. Novak, J. Sohl-Dickstein, and J. Pennington, *Wide neural networks of any depth evolve as linear models under gradient descent*, Adv. Neural Inform. Process. Syst., 32 (2019).

[21] C. Liu, L. Zhu, and M. Belkin, *On the linearity of large non-linear models: When and why the tangent kernel is constant*, Adv. Neural Inform. Process. Syst., 33 (2020), pp. 15954–15964.

[22] C. Liu, L. Zhu, and M. Belkin, *Toward a Theory of Optimization for Over-parameterized Systems of Non-linear Equations: The Lessons of Deep Learning*, preprint, https://doi.org/10.48550/arXiv.2003.00307, 2020.

[23] P. M. Long, *Properties of the After Kernel*, preprint, arXiv:2105.10585, 2021.

[24] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*, Nature Mach. Intell., 3 (2021), pp. 218–229, https://doi.org/10.1038/s42256-021-00302-5.

[25] G. Meanti, L. Carratino, L. Rosasco, and A. Rudi, *Kernel methods through the roof: Handling billions of points efficiently*, Adv. Neural Inform. Process. Syst., 33 (2020), pp. 14410–14422.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

[26] T. Mikolov, K. Chen, G. Corrado, and J. Dean, *Efficient Estimation of Word Representations in Vector Space*, preprint, arXiv:1301.3781, 2013.

[27] R. Novak, J. Sohl-Dickstein, and S. S. Schoenholz, *Fast finite width neural tangent kernel*, in Proceedings of the International Conference on Machine Learning, PMLR, 2022, pp. 17018–17044.

[28] R. Novak, L. Xiao, J. Lee, Y. Bahri, G. Yang, J. Hron, D. A. Abolafia, J. Pennington, and J. Sohl-Dickstein, *Bayesian Deep Convolutional Networks with Many Channels Are Gaussian Processes*, preprint, arXiv:1810.05148, 2018.

[29] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay, *The RefinedWeb dataset for falcon LLM: Outperforming curated corpora with web data, and web data only*, Adv. Neural Inf. Process. Syst., 36 (2023), pp. 79155–79172.

[30] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, *Improving Language Understanding by Generative Pre-training*, https://api.semanticscholar.org/CorpusID:49313245, 2018.

[31] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, *Language Models Are Unsupervised Multitask Learners*, https://api.semanticscholar.org/CorpusID:160025533, 2019.

[32] A. Radhakrishnan, D. Beaglehole, P. Pandit, and M. Belkin, *Mechanism for feature learning in neural networks and backpropagation-free machine learning models*, Science, 383 (2024), pp. 1461–1467, https://doi.org/10.1126/science.adi5639.

[33] B. Schölkopf, R. Herbrich, and A. J. Smola, *A generalized representer theorem*, in Proceedings of the International Conference on Computational Learning Theory, Springer, 2001, pp. 416–426.

[34] B. Schölkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*, MIT Press, 2002.

[35] H. Touvron, L. Martin, K. R. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. M. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. S. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. M. Kloumann, A. V. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, *Llama 2: Open Foundation and Fine-Tuned Chat Models*, preprint, https://arxiv.org/abs/2307.09288, 2023.

[36] M. Vargas, A. Tsou, A. Engel, and T. Chiang, *Foundation Model's Embedded Representations May Detect Distribution Shift*, preprint, https://arxiv.org/abs/2310.13836, 2024.

[37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, *Attention is all you need*, Adv. Neural Inform. Process. Syst., 30 (2017).

[38] S. Wang, H. Wang, and P. Perdikaris, *Learning the solution operator of parametric partial differential equations with physics-informed DeepONets*, Sci. Adv., 7 (2021), eabi8605, https://doi.org/10.1126/sciadv.abi8605.

[39] S. Wang, H. Wang, and P. Perdikaris, *Improved architectures and training algorithms for deep operator networks*, J. Sci. Comput., 92 (2022), 35, https://doi.org/10.1007/s10915-022-01881-0.

[40] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, and J. Brew, *Huggingface's Transformers: State-of-the-Art Natural Language Processing*, preprint, http://arxiv.org/abs/1910.03771, 2019.

[41] D. Yarotsky, *Optimal approximation of continuous functions by very deep ReLU networks*, in Proceedings of the Conference on Learning Theory, PMLR, 2018, pp. 639–649.

Downloaded 08/10/26 to 47.42.132.40 by Saad Qadeer (saad21290@gmail.com). Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.